

LUT TENSOR CORE: A Software-Hardware Co-Design for LUT-Based Low-Bit LLM Inference

Zhiwen Mo*
Imperial College London and
Microsoft Research
London, UK

Lei Wang*
Peking University and
Microsoft Research
Beijing, China

Jianyu Wei*
University of Science and Technology
of China and Microsoft Research
Beijing, China

Zhichen Zeng*
University of Washington and
Microsoft Research
Seattle, USA

Shijie Cao[†]
Microsoft Research
Beijing, China

Lingxiao Ma
Microsoft Research
Beijing, China

Naifeng Jing
Shanghai Jiao Tong University
Shanghai, China

Ting Cao
Microsoft Research
Beijing, China

Jilong Xue
Microsoft Research
Beijing, China

Fan Yang
Microsoft Research
Beijing, China

Mao Yang
Microsoft Research
Beijing, China

Abstract

Large Language Model (LLM) inference becomes resource-intensive, prompting a shift toward low-bit model weights to reduce the memory footprint and improve efficiency. Such low-bit LLMs necessitate the mixed-precision matrix multiplication (mpGEMM), an important yet underexplored operation involving the multiplication of lower-precision weights with higher-precision activations. Off-the-shelf hardware does not support this operation natively, leading to indirect, thus inefficient, dequantization-based implementations.

In this paper, we study the lookup table (LUT)-based approach for mpGEMM and find that a conventional LUT implementation fails to achieve the promised gains. To unlock the full potential of LUT-based mpGEMM, we propose LUT TENSOR CORE, a software-hardware co-design for low-bit LLM inference. LUT TENSOR CORE differentiates itself from conventional LUT designs through: 1) software-based optimizations to minimize table precompute overhead and weight reinterpretation to reduce table storage; 2) a LUT-based Tensor Core hardware design with an elongated tiling shape to maximize table reuse and a bit-serial design to support diverse precision combinations in mpGEMM; 3) a new instruction set and compilation optimizations for LUT-based mpGEMM. LUT TENSOR CORE significantly outperforms existing pure software LUT implementations and achieves a 1.44× improvement in compute density and energy efficiency compared to previous state-of-the-art LUT-based accelerators.

*Work is done during internship at Microsoft Research.

[†]Corresponding Author

CCS Concepts

• **Computer systems organization** → **Neural networks; Architectures**; • **Hardware** → **Arithmetic and datapath circuits**.

Keywords

Low-bit LLM, Software-hardware co-design, LUT, Accelerator

ACM Reference Format:

Zhiwen Mo, Lei Wang, Jianyu Wei, Zhichen Zeng, Shijie Cao, Lingxiao Ma, Naifeng Jing, Ting Cao, Jilong Xue, Fan Yang, and Mao Yang. 2025. LUT TENSOR CORE: A Software-Hardware Co-Design for LUT-Based Low-Bit LLM Inference. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3695053.3731057>

1 Introduction

The advent of Large Language Models (LLMs) offers transformative opportunities across various AI applications [1, 3, 28, 65]. However, the deployment of LLMs requires substantial hardware resources [21, 54, 55]. To reduce inference costs, low-bit LLMs have emerged as promising approaches [10, 15, 31, 40]. Among different solutions, weight quantization, i.e., quantizing LLMs with low-precision weights and high-precision activations, has become particularly attractive as it reduces memory and computation costs while maintaining model accuracy [39, 75, 81]. While 4-bit weight quantization has become pervasive [12, 32, 64], both academia and industry are actively exploring advancements toward 2-bit and even 1-bit to further improve efficiency [4, 14, 29, 42, 44, 49, 68].

Weight quantization shifts the key computation pattern of LLM inference from conventional General Matrix Multiplication (GEMM) to **mixed-precision GEMM (mpGEMM)**, where one input matrix is in lower precision (e.g., INT4/2/1 weights) and the other remains in higher precision (e.g., FP16/8, INT8 activations). Currently, off-the-shelf hardware does not support mixed-precision operations



natively. Consequently, most low-bit LLM inference systems have to utilize *dequantization-based* approaches for mpGEMM [16, 39, 51, 69]. Dequantization upscales low-bit representations to match the hardware-supported GEMM. Such extra operations can become a performance bottleneck in large batch scenarios.

Lookup table (LUT) is another popular approach for low-bit computation and is well-suited for mpGEMM [26, 38, 45, 53, 71]. By precomputing multiplication results between low-precision weights and high-precision activations, LUT-based methods eliminate the need for dequantization and replace complex operations with simple table lookups. In practice, LUTs are implemented on a per-tile basis. For each small tile of mpGEMM, a lookup table is precomputed specifically for the activations within a tile and reused across weight matrix columns, significantly reducing storage overhead while maintaining efficiency.

Despite its potential, LUT-based mpGEMM still experiences notable performance gaps and challenges in both software and hardware implementations. On the software side, LUT kernels face limited instruction support and inefficient memory access, which lead to suboptimal performance compared to dequantization-based kernels on GPUs, as shown in Figure 4. On the hardware side, conventional LUT designs lack optimization for mpGEMM and often fall short of expected performance improvements. This is due to key challenges such as high table precomputation and storage overhead, limited support for diverse bit-width combinations, inefficiencies from suboptimal tiling shapes, and the lack of dedicated instruction sets and compilation support; see §2.3 for details.

LUT TENSOR CORE addresses these challenges through a holistic software and hardware co-design. By optimizing hardware-unfriendly tasks, such as table precomputation and storage management, in a software-based approach, LUT TENSOR CORE reduces the workload on hardware, simplifying its design and improving its compactness and efficiency. To be specific:

Software optimization (§ 3.1). To amortize the overhead of precomputing lookup tables, we observed that conventional designs precompute redundantly across multiple units. LUT TENSOR CORE splits the precomputation into an independent operator, thus avoiding redundancies, and fuses it with the previous operator to further reduce memory accesses. To reduce storage overhead, LUT TENSOR CORE exposes and exploits the inherent symmetry of a lookup table for mpGEMM by reinterpreting $\{0, 1\}$ as $\{-1, 1\}$, reducing the table size by half. LUT TENSOR CORE also reduces the table width and supports various activation bit widths by applying table quantization techniques.

Hardware customization (§ 3.2). LUT TENSOR CORE customizes the LUT-based Tensor Core design. The aforementioned software optimizations have simplified the hardware design by offloading the circuitry tasks to software, reducing the need for broadcasting and multiplexers by half. Meanwhile, LUT TENSOR CORE incorporates a flexible bit-serial-like circuit to accommodate various combinations of mixed precision operations. Moreover, LUT TENSOR CORE conducts a design space exploration (DSE) for the shape of LUT-based Tensor Core and identifies an elongated tiling shape that enables more efficient table reuse.

New instruction and compilation support (§ 3.3). LUT TENSOR CORE extends the traditional Matrix Multiply-Accumulate (MMA) instruction set to the LUT-based Matrix Multiply-Accumulate (LMMA)

instruction set, which includes essential metadata specifying the operand types and shapes. With the extension, LUT TENSOR CORE leverages the shape information provided in LMMA to recompile LLM workloads using tile-based deep learning compilers [5, 62, 84], producing efficient kernels for the new hardware.

Our LUT-based Tensor Core exhibits a power and area reduction of $4\times$ to $6\times$ compared to the conventional Tensor Core. To validate the performance enhancement of mpGEMM, we integrate our LUT-based Tensor Core design and instructions into Accel-Sim [30], a GPU hardware simulator. The results show that our LUT-based Tensor Core occupies only 16% of the area of a conventional Tensor Core while achieving even higher mpGEMM performance. Compared to state-of-the-art (SOTA) LUT software implementations [53], LUT TENSOR CORE achieves up to a $1.42\times$ speedup in general matrix vector multiplications (GEMV) and a $72.2\times$ speedup in GEMM. Compared to SOTA LUT accelerators [38], LUT TENSOR CORE achieves $1.44\times$ higher compute density and energy efficiency, enabled by the software-hardware co-design. Our code is available at https://github.com/microsoft/T-MAC/tree/LUTTensorCore_ISCA25.

Our contributions can be summarized as follows:

- We propose LUT TENSOR CORE, a software-hardware co-design for LUT-based mpGEMM to boost the inference efficiency of low-bit LLMs.
- Experiments show that the proposed LUT-based Tensor Core achieves $4\times$ to $6\times$ power, performance, and area (PPA) gains. LUT TENSOR CORE exhibits inference speedups of $2.06\times$ to $5.51\times$ for low-bit LLMs like BitNet and quantized LLAMA models, with comparable area and accuracy.
- Beyond efficiency, our design can accommodate a wide range of weight (e.g., INT4/2/1) and activation precisions (e.g., FP16/8, INT8). Moreover, LUT TENSOR CORE can be integrated into existing inference hardware and software stacks with the extended LMMA instructions and compilation optimizations.

2 Background and Motivation

2.1 LLM Inference and Low-Bit Quantization

Recently, LLMs mainly rely on the decoder-only transformer architecture [66], as shown in Figure 1. Specifically, LLMs are built with sequential transformer layers, where each transformer layer contains a multi-head attention block followed by a feed-forward block. In both blocks, the primary computations are GEMM, or mpGEMM operations with weight quantization. The scaling up of LLMs requires substantial hardware resources [21, 28]. For example, LLAMA-2-70B [65] consumes 140GB of memory for its model weights alone (in FP16), far exceeding the capacity of a modern GPU like NVIDIA A100 or H100. This imposes a considerable challenge for LLM deployment.

To reduce inference costs in LLM deployment, low-bit quantization has become a popular approach [10, 12, 64, 76]. It reduces the precision of numerical representations of a model, thus decreasing memory footprint and computation time. In LLM quantization, *weight quantization* is preferred over activation quantization [37, 39]. This is because the values of model weights are static and thus can be quantized offline. Weights can be quantized to 4-bit, 2-bit, and even 1-bit. Post-training quantization (PTQ) incurs

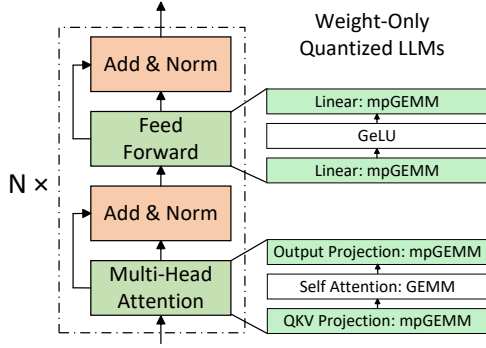


Figure 1: Decoder-only transformer blocks in LLMs. The primary computations are GEMM operations (or mpGEMM operations with weight quantization).

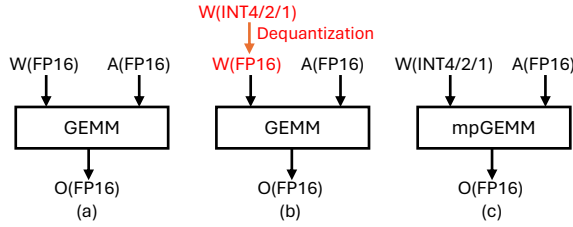


Figure 2: (a) GEMM, (b) Indirect mpGEMM with dequantization, (c) Direct mpGEMM for low-bit LLM inference.

minimal accuracy loss for 4-bit weights [12, 64, 76]. Recent studies and practices show that 2-bit weight quantization outperforms 4-bit in model accuracy at the same memory budget using quantization-aware training (QAT) [14, 42, 49]. BitNet further shows that training models with 1.58-bit (ternary) or even 1-bit (binary) weights from scratch can achieve comparable accuracy with 16-bit models [44, 68]. ParetoQ[42] also reports 2-bit quantization offers promising potential for memory reduction and speedup considering hardware constraints.

Conversely, activations are generated on-the-fly with high variance, presented as dynamic outliers [10, 18, 73]. Due to the presence of outliers, it is challenging to quantize activations below 8 bits. Different combinations of weight and activation bit-widths have been explored across various models and scenarios [10, 14, 15, 19, 68], suggesting that no universal solution fits all scenarios.

2.2 LUT-based mpGEMM for Low-Bit LLM

The varying bit-widths of weights and activations lead to a unique requirement of mixed-precision GEMM (mpGEMM), such as INT4/2/1 multiplied by FP16, illustrated in Figure 2. Current commercial LLM inference hardware, such as GPUs and TPUs, lack native support for mpGEMM, focusing instead on conventional GEMM with uniform input formats. **Dequantization-based mpGEMM** bridges this gap by upscaling low-precision weights to match high-precision activations [50, 69]. However, this approach introduces additional dequantization operations and forgoes the efficiency gains of low-precision computation.

LUT-based mpGEMM is an increasingly attractive approach for low-bit LLM inference [26, 38, 45, 53, 71]. It precomputes dot

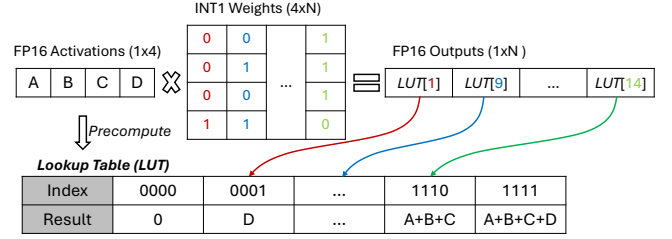


Figure 3: A naive LUT-based mpGEMM tile example of FP16 activations and INT1 weights. With the precomputed table, a table lookup can replace a dot product of 4-element vectors.

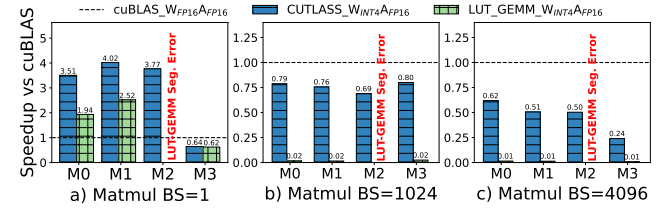


Figure 4: mpGEMM kernel performance with shapes M0-M3 extracted from LLAMA2-70B. $W_{INT4 \times A_{FP16}}$ denotes INT4 weights and FP16 activations. LUT-based software kernels (LUT-GEMM) underperform dequantization-based kernels (CUTLASS) on the A100 GPU.

products between high-precision activations and low-precision weights, which are then stored in lookup tables (LUT) for fast retrieval. Instead of precomputing a massive table for all possible combinations of high-precision and low-precision values (e.g., $FP16 \times INT4$, which would require a table of size $(2^{16} \times 2^4)$), LUT-based mpGEMM organizes computations in a tiled manner. For each small tile of the mpGEMM operation, i.e., each small group of activations, a LUT is precomputed specifically for these activation values and reused across weight columns. This approach minimizes table size and maintains efficiency by dynamically building LUTs for each tile during computation. Figure 3 illustrates a basic example where a small tile consists of 1x4 FP16 activations and 4xN INT1 weights. With an activation vector length of 4, the lookup table size is 16. In this case, each result of the dot product of length 4 can be obtained through a simple table lookup. The table can be reused N times, which is significant given the size of the weight matrix. Larger activation vectors or higher-bit weights require proportionally larger lookup tables.

2.3 Gaps in Current LUT-based Solutions

LUT-based mpGEMM is promising due to its advantages in eliminating dequantization and multiplication and reducing additions through simple table lookups. However, existing software and hardware implementations face challenges and gaps:

Software LUT kernel. LUT-based mpGEMM software kernels often face challenges related to limited instruction support and inefficient memory access. The limitations are two-fold: First, GPU instruction support for table lookups is limited. The most effective available instruction, prmt (permute), has a limited width that

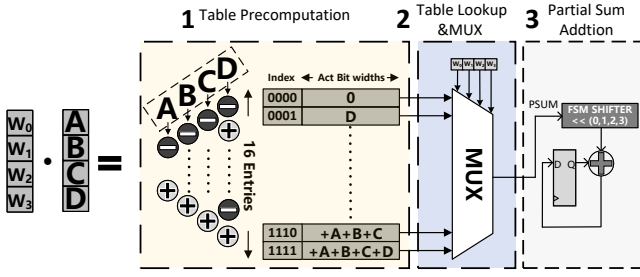


Figure 5: Conventional LUT hardware in three steps. Table precomputation and storage introduce heavy overhead.

prevents completing a whole table lookup in a single instruction, reducing throughput. Second, table location significantly affects performance. Storing lookup tables in the register file causes extensive data duplication across threads due to the broadcast nature of LUT methods, leading to register spillage when handling large tables. Conversely, placing tables in shared memory may result in bank conflicts due to random accesses by threads within a warp, severely affecting memory bandwidth. These issues lead to their reduced effectiveness compared to dequantization-based kernels on existing LLM inference hardware, such as GPUs. Figure 4 compares the performance of the LUT-based mpGEMM kernel in [53] to the dequantization-based mpGEMM kernel in CUTLASS [50] on A100 GPU. The results indicate that the dequantization-based kernel consistently outperforms the LUT-based kernel. Notably, when the batch size is large, the LUT-based kernel suffers from significant performance degradation due to table access overhead, performing several orders of magnitude worse. The “Seg. Error” annotation indicates a segmentation error observed in LUT-GEMM[53].

Hardware LUT Accelerator. At first glance, customized LUT hardware promises efficiency gains due to its simplicity, requiring only registers for table storage and multiplexers for lookups. However, our study indicates that conventional LUT hardware designs fall short of delivering these gains. Figure 5 depicts a conventional three-step LUT-based hardware solution for mpGEMM: table precomputation, table lookup, and partial sum addition. Numerous challenges and unexplored design aspects significantly impact the overall performance: (1) Table precompute and storage. Precomputed tables can occupy excessive storage, incurring area and latency overhead and thus diminishing efficiency gains. (2) Bit-width flexibility. Supporting various bit-width combinations (e.g., INT4/2/1 $\times$ FP16/FP8/INT8) may consume excessive chip area. (3) LUT tiling shape. Suboptimal tiling increases storage costs and limits table reuse opportunities, impacting performance. (4) Instruction and compilation. A new instruction set is required for LUT-based mpGEMM. However, the conventional compilation stack, optimized for standard GEMM hardware, may not efficiently map and schedule the new instruction set, complicating integration with existing software stacks.

3 LUT TENSOR CORE Design

We introduce LUT TENSOR CORE, a software-hardware co-design aimed at addressing the aforementioned efficiency, flexibility, and integration challenges (§2.3). Figure 6 provides an overview of LUT

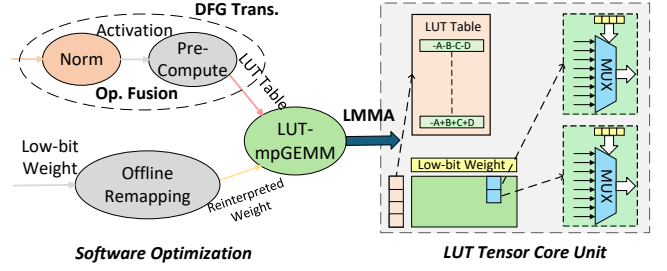


Figure 6: LUT TENSOR CORE workflow.

TENSOR CORE. Different from conventional hardware-based LUT solutions where table precompute and storage introduce significant hardware overhead, LUT TENSOR CORE introduces software-based optimizations (§3.1) to optimize the table precompute and storage: precomputing the LUT table for the input activation tensor is performed by operator fusion, while the input weight tensor is reinterpreted to enable table storage optimizations. On the hardware side, the LUT-based Tensor Core microarchitecture (§3.2) provides efficiency for mpGEMM processing and flexibility for different bit-width data types. To integrate LUT TENSOR CORE into existing deep learning ecosystem, LUT TENSOR CORE designs the LMMA instruction set to expose the LUT-based Tensor Core for programming mpGEMMs and implements a compilation stack to schedule the end-to-end LLM execution (§3.3).

3.1 Software-based Table Optimization

As introduced in §2, LUT-based mpGEMM requires an additional table precomputation process and storage to store the precomputed results. Naively, the precomputed dot products of a length K activation vector on the W_BIT weight require $(2^{W_BIT})^K$ entries for the table. For each activation element, multiplying it with the W_BIT weight has 2^{W_BIT} possible results, constructing the precompute table for this activation element. Therefore, the precomputed table has $(2^{W_BIT})^K$ entries for a length K activation vector. Figure 3 shows the lookup table with 2^4 entries for $K = 4$, $W_BIT = 1$.

A commonly-used optimization is bit-serial [27], which represents a W_BIT integer as W 1-bit integers and performs multiplication over 1-bit integers with bit shift. This paradigm can reuse the precompute table on 1-bit, and therefore reduces the table size to 2^K . Nonetheless, even this reduced table size entails significant hardware overhead. LUT TENSOR CORE proposes dataflow graph (DFG) transformation and operator fusion to eliminate the table precomputation overhead, as well as weight reinterpretation and table quantization to reduce the table size.

3.1.1 Precomputing lookup table with DFG transformation and operator fusion.

The LUT-based mpGEMM requires precomputing the dot products between high-precision activations and a set of low-precision weights as a table for the later lookup operations. Conventional implementations position the precompute unit adjacent to the LUT unit, performing table precomputation on-the-fly for each LUT unit. This approach introduces significant hardware costs due to redundancy, as multiple precompute units often perform identical operations. Considering an example of $[4096, 12288] \times [12288, 12288]$ GEMM in OPT-175B, a naive direct

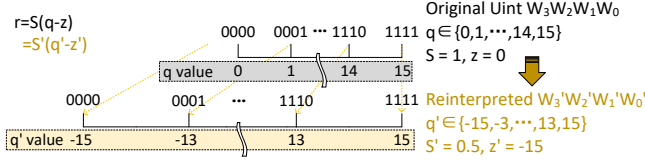


Figure 7: Reinterpreting 0,1 to -1,1 to enable symmetry, thereby cutting the table size by half.

precompute unit shares a table across a LUT-based Tensor Core within an array size of $N=4$. In this setup, each table is computed repeatedly ($12288/4 = 3072$ times) by different LUT units throughout the process, imposing a significant computational burden.

To address this inefficiency, we first transform the DFG to split the precomputation into an independent kernel, enabling one-time precomputation that can be broadcasted to all LUT units. This modification reduces the precomputation overhead by hundreds of times, making it manageable by existing vector units like CUDA Cores. To amortize the additional memory traffic introduced by broadcasting, LUT TENSOR CORE fuses the precompute operator with the preceding operator, leveraging its element-wise computation pattern, as shown in Figure 6 and detailed in §3.3.2. This fusion reduces memory access and brings precomputation overhead down to almost zero, as evaluated in §4.6.1.

3.1.2 Reinterpreting weight for table symmetrization. The 2^K table size of precomputing a length K activation vector introduces a significant cost in both table storage and table accesses. To address this issue, we observed and leveraged the symmetrization property of the integer representation.

Assume that the originally quantized weights are represented as:

$$r_w = s_w(q_w - z_w) \quad (1)$$

where r_w is the real-valued weight, s_w is the scale factor, z_w is the bias, and q_w is the K -bit integer representation.

Our goal is to map q_w such that it is symmetric around zero while maintaining mathematical equivalence. To achieve this, both s_w and z_w must be adjusted. When mapping a uint q_w to be symmetric about zero, the following adjustments are required:

$$q'_w = 2q_w - (2^K - 1), \quad s'_w = s_w/2, \quad z'_w = 2z_w + 1 - 2^K \quad (2)$$

This process is illustrated in Figure 7, showing an example of transforming 4-bit unsigned integers. By calculating s'_w and z'_w , q'_w is mapped from $\{0, 1, \dots, 14, 15\}$ to $\{-15, -13, \dots, 13, 15\}$, achieving symmetry around zero.

Next, the dot product can be represented as:

$$DP = \sum Act_i s_w(q_{wi} - z_w) = \sum Act_i s'_w(q'_{wi} - z'_w) \quad (3)$$

where DP is the dot product and Act_i is the activation value. Therefore, the quantization process remains the same as before, with the additional step of an offline mapping for the weight's $s_w(q_{wi} - z_w)$ to $s'_w(q'_{wi} - z'_w)$. Let us consider a dot product between the binary representation $W_3W_2W_1W_0 = 0100$ and variables A, B, C, D . Initially, the binary values $\{0, 1\}$ are interpreted as $\{0, 1\}$. Assume $s_w = 2$

and $z = 1/2$. The calculation proceeds as follows:

$$\begin{aligned} DP &= \sum Act_i s_w(q_{wi} - z_w) \\ &= A \cdot 2 \cdot (0 - 0.5) + B \cdot 2 \cdot (1 - 0.5) \\ &\quad + C \cdot 2 \cdot (1 - 0.5) + D \cdot 2 \cdot (1 - 0.5) \\ &= -A + B - C - D \end{aligned}$$

After reinterpretation, the binary values $\{0, 1\}$ are remapped to represent $\{-1, 1\}$, with adjusted scale factor $s'_w = 1$ and bias $z'_w = 0$. The updated computation is:

$$\begin{aligned} DP &= \sum Act_i s'_w(q'_{wi} - z'_w) \\ &= A \cdot 1 \cdot (-1 - 0) + B \cdot 1 \cdot (1 - 0) \\ &\quad + C \cdot 1 \cdot (-1 - 0) + D \cdot 1 \cdot (-1 - 0) \\ &= -A + B - C - D \end{aligned}$$

It is clear that the two expressions remain mathematically equivalent. As the table entries are symmetric about zero, the lookup table exhibits properties similar to odd functions. Assuming the index is a 4-bit value $W_3W_2W_1W_0$, a naive implementation of the lookup table (LUT) requires $2^4 = 16$ entries. However, it can be observed that the following property, akin to that of odd functions, holds:

$$\text{LUT}[W_3W_2W_1W_0] = -\text{LUT}[\sim(W_3W_2W_1W_0)] \quad (4)$$

Therefore, the number of entries in the LUT can be reduced to half of the original, which is $2^{4-1} = 8$, and the equation becomes:

$$\text{LUT}[W_3W_2W_1W_0] = \begin{cases} -\text{LUT}[\sim(W_2W_1W_0)], & \text{if } W_3 = 1 \\ \text{LUT}[W_2W_1W_0], & \text{if } W_3 = 0 \end{cases} \quad (5)$$

Here, $\sim$ denotes the bit-wise NOT operation. Therefore, given a length K activation vector, table symmetrization can reduce the table length to 2^{K-1} . The table size not only affects the computational operations required during the precompute stage, but also the multiplexers' size. Furthermore, each entry in the table also needs to be broadcast to N PEs, typically 64 or 128, for dot product computations. Such an optimization significantly reduces the broadcasting overhead and the MUX selection overhead. Note that $W_3W_2W_1W_0$ in Equation 5 are static weights. The bit-level negation can be done offline to simplify the design as follows:

$$\text{LUT}[W'_3W'_2W'_1W'_0] = \begin{cases} -\text{LUT}[W'_2W'_1W'_0], & \text{if } W'_3 = 1 \\ \text{LUT}[W'_2W'_1W'_0], & \text{if } W'_3 = 0 \end{cases} \quad (6)$$

This simplification can eliminate the negation operation in circuit design, which will be introduced in §3.2.

3.1.3 Table quantization. For high-precision activations such as FP32 or FP16, we employ table quantization techniques to convert the precomputed table elements to a lower, unified precision like INT8. This approach offers flexibility through support for multiple activation precisions and improves efficiency by reducing table size.

Compared to traditional activation quantization, table quantization allows for more dynamic, fine-grained quantization during the table precomputation stage. For example, with a group size of 4 activation elements, we perform quantization for each table that contains 8 precomputed dot products. Our empirical experiments, discussed in §4.6.2, demonstrate that INT8 table quantization maintains high accuracy while simplifying hardware design, thereby validating the effectiveness of our approach.

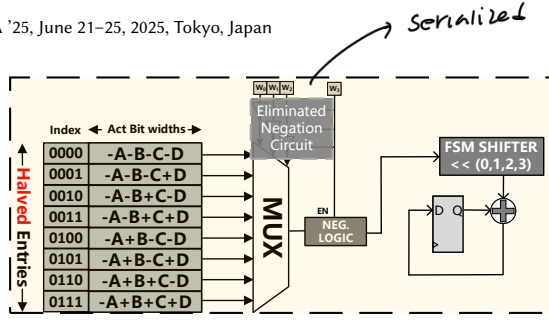


Figure 8: Optimized LUT unit with bit-serial.

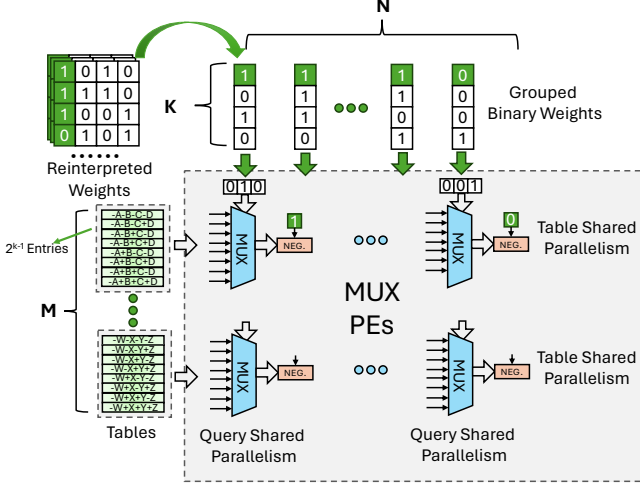


Figure 9: Elongated MNK tiling of LUT-based Tensor Core. LUT-based Tensor Core requires a larger N (e.g., 64/128) to maximize table reuse, along with a suitably sized K (e.g., 4) for a cost-efficient table size.

3.2 LUT-based Tensor Core Microarchitecture

3.2.1 Simplified LUT unit design with bit-serial. By leveraging software-based precompute fusion and weight reinterpretation, the hardware cost for customizing each individual LUT unit is reduced. Figure 8 illustrates our LUT unit design. Compared to a straightforward design, the registers required for storing the LUT and the costs associated with table broadcasting and multiplexers are reduced by half. As shown in Equation 6, the bit-level negation circuit can be eliminated from each LUT unit to further improve efficiency. To support flexible bit-widths for weights, we employ a bit-serial circuit architecture [27, 74]. This design maps the weight bit-width to W_BIT cycles, enabling the processing of various bit-widths in a serialized manner.

3.2.2 Elongated LUT tiling. The selection of dimensions M , N , and K is crucial for the performance of the LUT-based Tensor Core, as traditional choices for MAC-based Tensor Cores may result in suboptimal performance. As illustrated in Figure 9, a MNK Tile's LUT Array comprises M tables, N sets of weights, and $M \times N$ MUX-based units. Each table contains $M \times 2^{K-1}$ entries, with each entry broadcast to N MUX units. Each set of *Grouped Binary Weights* includes K bits, which must be broadcast to M MUX units to act as select signals for the MUX. The total table size is given by the equation:

$$\text{Total Table Size} = M \times 2^{K-1} \times \text{LUT_BIT} \quad (7)$$

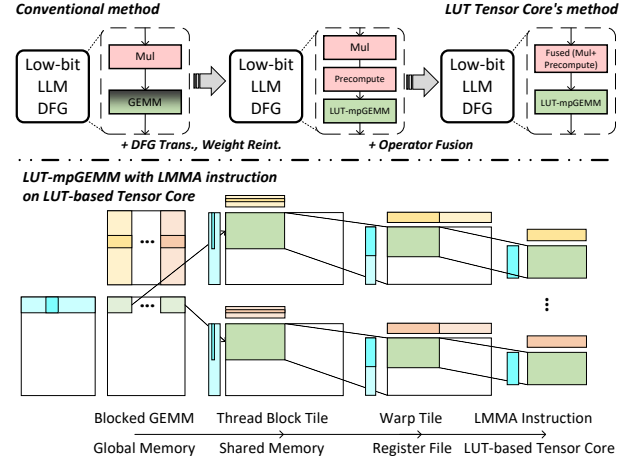


Figure 10: Compilation for LUT-mpGEMM. Overall dataflow is cutlass-like [50]. Elongated tile for better data reuse.

and the size for grouped binary weights is given by:

$$\text{Grouped Binary Weights Size} = K \times N \times W_BIT \quad (8)$$

where LUT_BIT is the bit width of the LUT entries, and W_BIT is the bit width of the weights.

An LUT-based Tensor Core benefits from an elongated tiling shape. When K is large, the number of table entries grows exponentially, whereas N determines how many MUX units can reuse each table entry. An optimal configuration requires a balanced K , a larger N , and a smaller M , unlike conventional GPU Tensor Cores. Additionally, the tiling shape affects I/O traffic, where a more square-like tiling configuration reduces data movement overhead. In §4.2.2, we explore the design space for MNK tiling, confirming that elongated tiling shapes yield higher efficiency.

3.3 Instruction and Compilation

To integrate LUT TENSOR CORE into existing GPU architectures and ecosystems, we introduce a new instruction set and develop a compilation stack based on tile-based DNN compilers [5, 62, 84].

3.3.1 LUT-based MMA instructions. To enable programming with LUT-based Tensor Core, we define a set of LMMA instructions as an extension of the MMA instruction set in GPU.

$$\text{lmma}\{M\}\{N\}\{K\}\{A_{dtype}\}\{W_{dtype}\}\{Accum_{dtype}\}\{O_{dtype}\}$$

The above formula shows the format of LMMA instructions, which resemble MMA. Specifically, the M , N , and K indicate the shape of the LUT-based Tensor Core. A_{dtype} , W_{dtype} , $Accum_{dtype}$, and O_{dtype} indicate the data types of the inputs, accumulation and the output, respectively. Similar to MMA instructions, each LMMA instruction is scheduled to a warp of threads for execution. Each warp calculates the formula $O_{dtype}[M, N] = A_{dtype}[M, K] \times W_{dtype}[N, K] + Accum_{dtype}[M, N]$.

3.3.2 Compilation support and optimizations. We implemented the LUT-mpGEMM kernel generation and end-to-end LLM compilation with LUT-based Tensor Core on top of TVM [5], Roller [84] and Welder [62]. Specifically, the compilation stack encompasses the

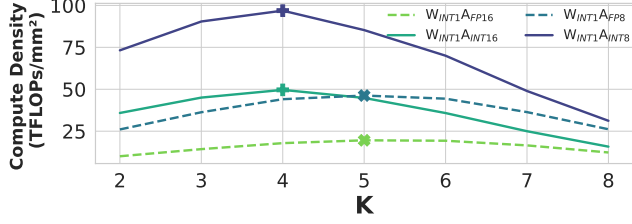


Figure 11: Design space exploration along the K-axis for the LUT-based dot product unit. $K = 4$ is the optimal in general.

following key aspects. Figure 10 shows an example of compilation on the LLAMA model:

- **DFG Transformation.** Given the model represented in DFG, we transform the mixed-precision GEMM operator to a precompute operator and a LUT-mpGEMM operator. This transformation is implemented as a graph optimization pass in Welder [62].
- **Operator Fusion.** Operator fusion is a widely-used compiler technique to optimize the end-to-end model execution by reducing memory traffic and runtime overhead. We reuse Welder for operator fusion by registering the precompute and LUT-mpGEMM operators with the required tile-based representation. As shown in Figure 10, the element-wise precompute operator is fused with the previous element-wise operator.
- **LUT-mpGEMM Scheduling.** Scheduling LUT-mpGEMM operator requires careful consideration of tiling in the memory hierarchy for optimal performance. Conventional GEMM tiling strategies [5, 82, 84] assume the same data type for both activations and weights. However, mpGEMM uses different data types for activation and weight, affecting memory transactions. To address this, we represent tiling by memory size rather than shape, and register LMMA instruction shapes and tiling calculations in Roller’s rTile [84] interfaces to schedule optimal configurations.
- **Code Generation.** With the finalized scheduling plans, code generation is performed using TVM. Specifically, the LMMA instructions are registered as intrinsics in TVM, and TVM can follow the scheduling to generate the kernel code with LMMA instructions.

4 Evaluation

In this section, we evaluate LUT TENSOR CORE to validate its efficiency in accelerating low-bit LLM inference. First, we assess the hardware efficiency gains of our design via detailed PPA benchmarking (§4.2). Then, kernel-level experiments are conducted to illustrate the acceleration of mpGEMM (§4.3). Next, we perform end-to-end inference evaluation on commonly-used LLMs to demonstrate the practical performance improvements (§4.4). Finally, we compare LUT TENSOR CORE with previous LUT-based works (§4.5) and evaluate the effectiveness of our software optimizations, focusing on table precompute fusion and table quantization (§4.6).

4.1 Experimental Setup and Methodology

4.1.1 Hardware PPA benchmarks. We compare our LUT-based Tensor Core with two baselines: Multiply-Accumulate (MAC)-based

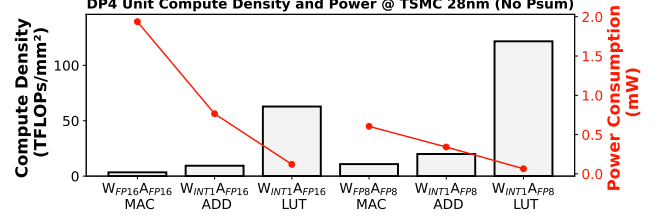


Figure 12: PPA comparison across MAC/ADD/LUT-based DP4 implementations. Our LUT-based DP4 unit has compute density and power advantages.

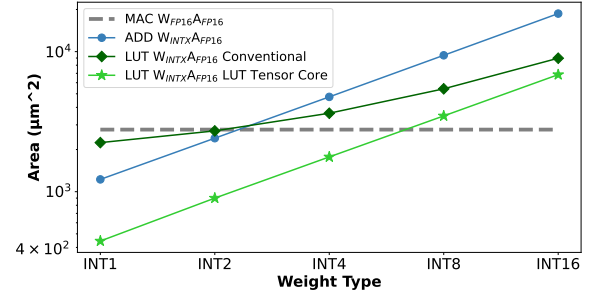


Figure 13: Area comparison of MAC, ADD, and LUT-based DP4 units across weight bit-widths in $W_{INTX} \times A_{FP16}$. Conventional LUT implementation does not have area advantages.

Tensor Core and Addition (ADD)-based Tensor Core. MAC represents the typical design in current GPUs which needs dequantization to support mpGEMM. ADD adopts the bit-serial computing proposed in [27] to support mpGEMM, where every bit of weights needs one addition. We implement LUT-based Tensor Core and baselines in Verilog and use Synopsys’s Design Compiler [63] and the TSMC 28nm process library for synthesizing circuits and generating PPA data. We apply DC’s medium effort level targeting 1GHz to ensure a fair comparison across all designs.

4.1.2 Kernel-level evaluation. For mpGEMM kernel-level evaluation, we use the NVIDIA A100 GPU as the baseline and employ Accel-Sim [30], an open-source state-of-the-art simulator. Modifications to the configuration and trace files in Accel-Sim enable us to simulate both the original A100 and the LUT TENSOR CORE-equipped A100.

4.1.3 Model end-to-end evaluation and analysis. To extend our evaluation to real LLMs, we utilize four widely-used open-source LLMs: LLAMA-2 [65], OPT [80], BLOOM [36], and BitNet [68]. Since Accel-Sim becomes infeasible for end-to-end LLM experiments due to its slow simulation speed for large trace files, we develop a tile-based simulator to support end-to-end inference evaluations, as detailed in §4.4.

4.2 Hardware PPA Benchmarks

4.2.1 Dot product unit microbenchmark. In this experiment, we fixed M and N to 1 and varied K (i.e., a dot product unit of K -element vectors) to explore its impact on compute density. A large K could lead to exponential growth in lookup table entries, whereas

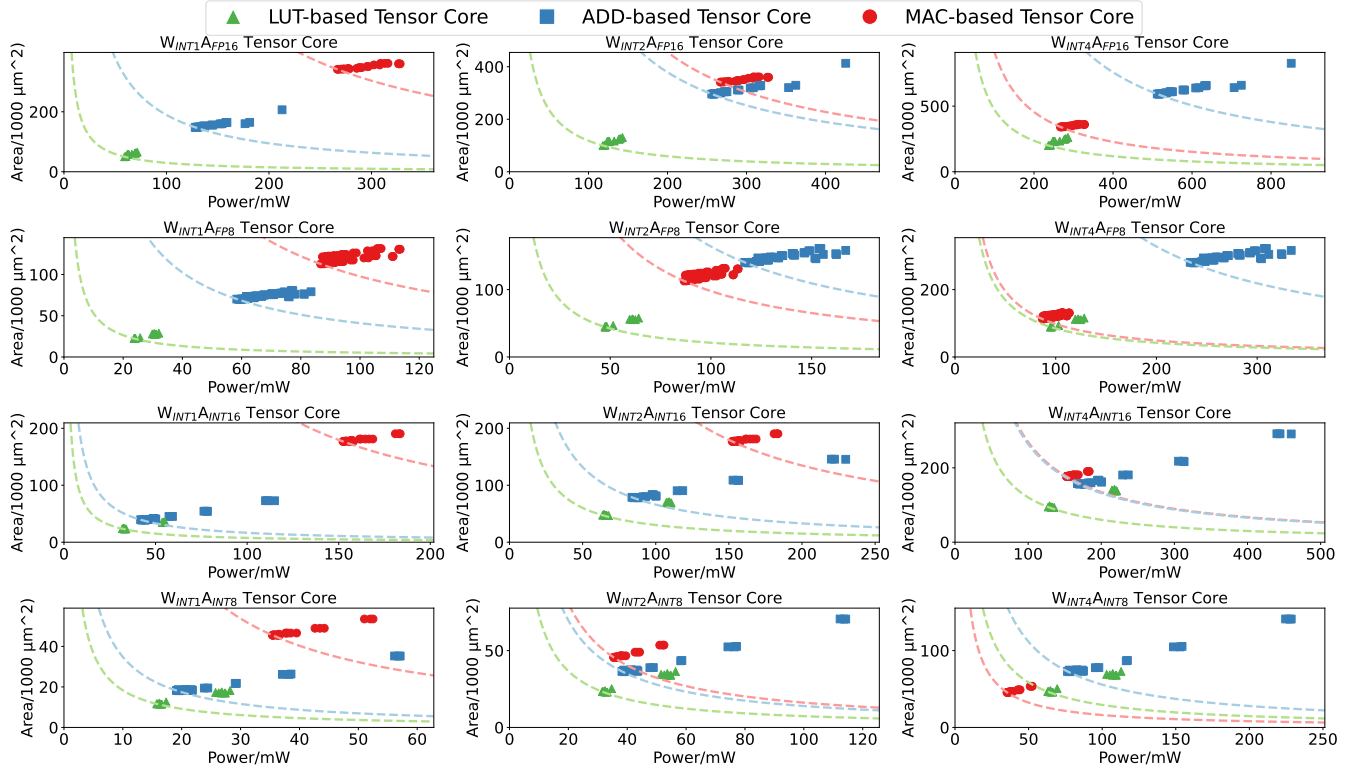


Figure 14: PPA across LUT-/ADD-/MAC-based Tensor Core implementations for mpGEMM.

a smaller K results in $1/K$ of the computations still being performed by adders. As shown in Figure 11, we found INT operations peak in density at $K = 4$, while floating-point operations perform best at $K = 5$ but also well at $K = 4$. Therefore, we adopt $K = 4$ for all subsequent LUT-based designs.

We conduct benchmarks on dot product implementations using MAC, ADD, and LUT-based approaches across various data formats. This includes uniform precision with MAC, such as $W_{FP16}A_{FP16}$, and mixed precision, such as $W_{INT1}A_{FP8}$, using both ADD and LUT approaches. As depicted in Figure 12, the LUT-based approach reaches 61.55 TFLOPs/mm² with $W_{INT1}A_{FP16}$, surpassing the conventional MAC implementation, which only registers 3.39 TFLOPs/mm² with $W_{FP16}A_{FP16}$. Power efficiency shows a similar trend, with LUT-based methods achieving higher efficiency than other approaches.

Furthermore, we conduct weight-bit scaling experiments for $W_{INTX} \times A_{FP16}$ DP4 units across MAC/ADD/LUT-based implementations. The experiments are configured with the Tensor Core’s N dimension set to 4 to match the A100’s configuration. As shown in Figure 13, the conventional LUT-based implementation does not have area advantages compared to the MAC baseline when the weight is more than 2 bits. The main area efficiency bottleneck is the table precompute and storage overhead. ADD-based implementations also only surpass the MAC baseline in the 1-bit and 2-bit cases. Through the software-hardware co-design, LUT TENSOR CORE outperforms all the baselines up to a weight bit-width of 6 and delivers better area efficiency compared to the conventional LUT implementation.

4.2.2 Tensor Core benchmark. We scale our evaluation to the Tensor Core level, incorporating a design space exploration to identify optimal MNK configurations. To match the configuration of the A100 INT8 Tensor Core with $M, N, K = 8, 4, 16$, we set our array size to $M \times N \times K = 512$. Our experiments involve various activation data types, including A_{FP16} , A_{INT16} , A_{FP8} , and A_{INT8} , as well as multiple weight bit-widths, such as W_{INT1} , W_{INT2} , and W_{INT4} . We compare the performance of our LUT-based approach with MAC- and ADD-based approaches.

As shown in Figure 14, we sweep different M, N, K configurations to explore the design space and ensure a fair comparison across all methods. The y-axis is labeled “area”, and the x-axis is labeled “power”. The dashed lines represent the contours where the minimum Area×Power point for each design methodology lies among all data points. Our results demonstrate that across 12 sets of experiments with different activation data formats and weight bit-widths, the LUT-based method achieves the smallest area and lowest power consumption, except in the $W_{INT8}A_{INT4}$ case. Notably, with 1-bit weights, the LUT-based approach exhibits a 4×-6× reduction in power and area compared to the MAC-based Tensor Core design. We identify the optimal MNK configuration for the LUT-based Tensor Core as $M2N64K4$. This result is due to the fact that activations are in high bits and weights are in low bits. Considering the overall bit-width, the M dimension calculates to $2 \times 16 = 32$ bits, while the N dimension computes to $64 \times 1 = 64$ bits. The overall bit configuration still approximates a square array.

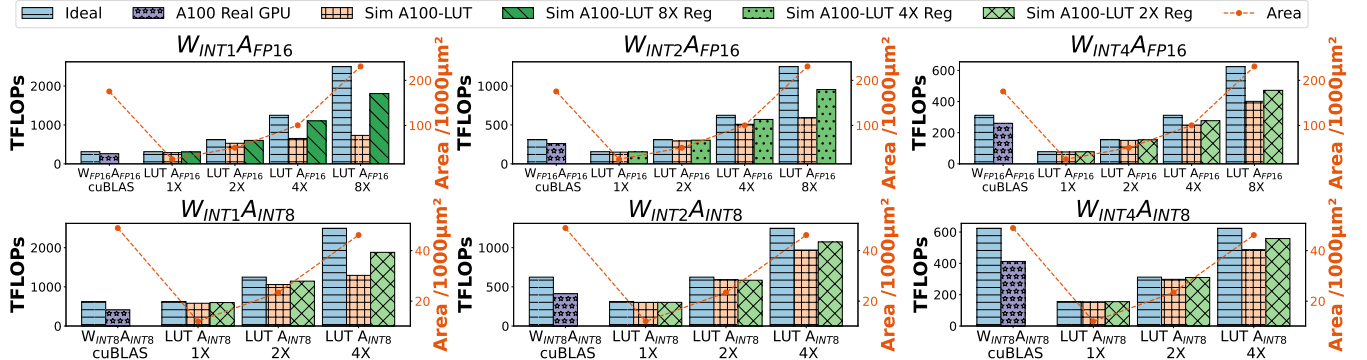


Figure 15: Accel-Sim runtime and area across A_{Fp16} and A_{Int8} Tensor Core designs. The symbol $\times$ denotes the Tensor Core array size relative to the 1 $\times$ baseline, where 1 $\times$ corresponds to the $M \times N \times K = 512$ array size in the NVIDIA A100.

4.3 mpGEMM Kernel-level Evaluation

We employ Accel-Sim, a SOTA GPU simulator, to validate the efficiency of LUT TENSOR CORE on mpGEMM operations and its compatibility with existing GPU architectures. The mpGEMM shape is extracted from LLAMA2-13B, with $M = 2048$, $N = 27648$, $K = 5120$. The dataflow of mpGEMM is designed to be cutlass-like and output-stationary, with tiling shapes optimized for efficient data reuse. For instance, a good candidate for $W_{Int8}A_{Int8}$ tiling sets the Thread Block tile to $[128, 512, 32]$ and the Warp tile to $[64, 256, 32]$.

As shown in Figure 15, LUT-based Tensor Core outperforms traditional MAC-based Tensor Core in mpGEMM operations. The leftmost two bars in each subplot represent A100’s ideal peak performance and the measured performance using cuBLAS. The remaining bars represent LUT-based results: ideal peak performance, simulated performance, and simulated performance with an increased register capacity. The register capacity adjustment addresses bottlenecks caused by insufficient registers, which restrict large tiling and tie performance to memory constraints. For example, with $W_{Int8}A_{Fp16}$, the LUT-based approach delivers slightly higher mpGEMM performance while using only 14.3% of the area of a MAC-based Tensor Core.

4.4 Model End-to-End Evaluation

While Accel-Sim offers detailed architectural emulation, it suffers from a slowdown of approximately five million times, transforming a ten-second task on an A100 GPU into a simulation period of up to 579 days, and generating trace files over 79TB in size.

To overcome these obstacles, we have developed an end-to-end simulator designed for rapid and accurate emulation with tile-level granularity. Our key insight is that the behavior of highly optimized, large GPU kernels with minimal stalling can be treated as accelerators, particularly in LLM scenarios. This perspective is supported by findings from NVIDIA in NVAS [67], which suggests viewing GPU simulation philosophically as “dynamically interacting roofline components”, rather than as a “cycle-by-cycle progression”. Accordingly, we adopt analytical methods from established accelerator modeling frameworks, such as Timeloop [52], Maestro [34], and Tileflow [83], to develop a tile-based GPU simulator. This tool facilitates a detailed and accurate evaluation of dataflow, memory

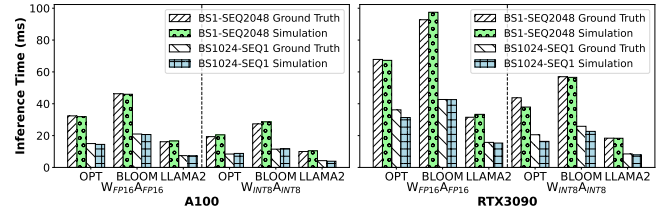


Figure 16: Evaluation of end-to-end simulator accuracy.

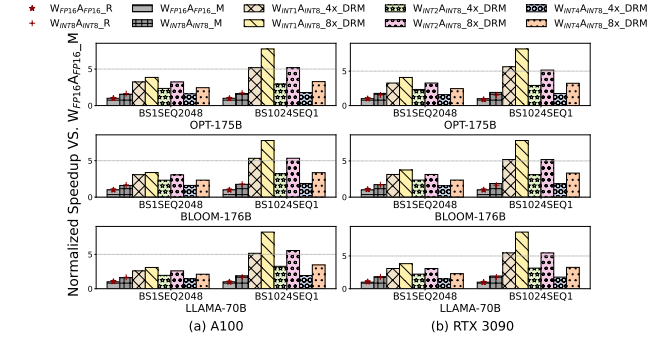


Figure 17: End-to-end simulation results on LLMs (A100 and 3090). R: Real GPU, M: Modeling, DRM: Double Reg Modeling.

bandwidth, computational resources, and operator fusion. We plan to open source this simulator in future work.

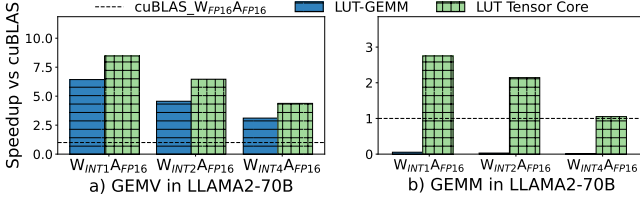
4.4.1 Simulator accuracy evaluation. In Figure 16, we validate our end-to-end simulator using OPT-175B, BLOOM-176B, and LLAMA2-70B, across various configurations on a single layer on both A100 and RTX 3090 GPUs. Our simulator achieves a mean absolute percentage error of only 5.21% against real GPU performance, while significantly faster than Accel-Sim in simulation speed.

4.4.2 End-to-end inference simulation results. Figure 17 presents benchmark results for the OPT, BLOOM, and LLAMA models. Our experiments demonstrate that LUT TENSOR CORE achieves an end-to-end speedup of up to 8.2 $\times$ while occupying less area compared to traditional $W_{Fp16}A_{Fp16}$ Tensor Cores. Notably, even under an 8 $\times$ setting, the area of LUT TENSOR CORE remains only 38.3% that of conventional $W_{Fp16}A_{Fp16}$ MAC-based Tensor Cores.

Table 1: Overall comparison.

HW. Config.	Model	Model Avg. Acc.	BS1 SEQ2048 Latency	BS1024 SEQ1 Latency	Peak Perf.	TC. Area Per SM	TC. Compute Density	TC. Energy Efficiency
A100 [†] FP16 TC.	LLAMA 3B ($W_{FP16}A_{FP16}$)	49.7%	106.71ms	41.15ms	312 TFLOPs	0.975mm ²	2.96 TFLOPs/mm ²	2.98 TFLOPs/W
A100 [†] INT8 TC	BitNet b1.58 3B ($W_{INT2}A_{INT8}$)	49.4%	67.06ms	21.70ms	624 TOPs	0.312mm ²	17.73 TOPs/mm ²	19.94 TOPs/W
A100 [†] -LUT-4X*	BitNet b1.58 3B ($W_{INT2}A_{INT8}$)	49.4%	42.49ms	11.41ms	1248 TOPs	0.187mm ²	61.84 TOPs/mm ²	33.32 TOPs/W
A100 [†] -LUT-8X*	BitNet b1.58 3B ($W_{INT2}A_{INT8}$)	49.4%	38.02ms	7.47ms	2496 TOPs	0.373mm ²	61.95 TOPs/mm ²	33.65 TOPs/W
H100 [†] FP8 TC	BitNet b1.58 3B ($W_{FP8}A_{FP8}$)	-	38.20ms	12.30ms	1525 TFLOPs	0.918mm ²	12.59TFLOPs/mm ²	12.24TFLOPs/W
H100 [†] -LUT-4X*	BitNet b1.58 3B ($W_{INT2}A_{FP8}$)	-	28.70ms	9.90ms	1525 TFLOPs	0.488mm ²	23.69TFLOPs/mm ²	16.35TFLOPs/W
H100 [†] -LUT-8X*	BitNet b1.58 3B ($W_{INT2}A_{FP8}$)	-	23.48ms	5.97ms	3049 TFLOPs	0.909mm ²	25.40TFLOPs/mm ²	17.32TFLOPs/W

Due to the lack of public data on A100/H100 Tensor Cores and their 7/4nm processes, [†] indicates that the data are normalized to 28nm at 1.41GHz and optimized to the best of our ability for fair comparison. -LUT* denotes LUT TENSOR CORE-equipped GPU with Double Register Modeling. × means that of A100 FP16 Tensor Core array size. TC. refers to Tensor Core. Model accuracy for A_{FP8} is not reported, as BitNet is trained from scratch in the A_{INT8} format. Prior works [33, 47, 81] show that A_{FP8} generally outperforms A_{INT8} in terms of accuracy.

**Figure 18: LUT TENSOR CORE compared with LUT-based software work LUT-GEMM [53] for GEMM and GEMV.**

4.4.3 Overall comparison. As shown in Table 1, the A100 equipped with LUT + BitNet delivers up to a 5.51× acceleration in inference speed while utilizing only 38.3% of the original Tensor Core’s area. This results in an increase of up to 20.9× in compute density and an 11.2× improvement in energy efficiency, enabled by the quantized LUT table and highly optimized LUT circuit through software-hardware co-design. Compared to the original $W_{FP8}A_{FP8}$ Tensor Core of H100, LUT TENSOR CORE can achieve up to a 2.02× improvement in area efficiency.

4.5 Compared to Prior Works

4.5.1 LUT-based software. LUT-GEMM [53] and T-MAC [71] are previous SOTA LUT-based software solutions for GPUs and CPUs, respectively. Since T-MAC is designed for CPUs, we use LUT-GEMM for a more relevant comparison on GPUs. LUT TENSOR CORE is configured using only 57.2% of the area of conventional FP16 Tensor Cores. Figure 18 presents the comparative speedups of LUT TENSOR CORE and LUT-GEMM relative to $W_{FP16}A_{FP16}$ cuBLAS on A100. LUT-GEMM improves performance only in GEMV cases, but is several dozen times slower in GEMM compared to cuBLAS. Compared to the software-based LUT-GEMM, LUT TENSOR CORE delivers up to 1.42× faster GEMV and 72.2× faster GEMM.

4.5.2 LUT-based hardware. UNPU [38] is the SOTA LUT-based hardware accelerator for DNN workloads. Since no public code is available, we re-implement the UNPU design based on its paper and apply optimizations to ensure a fair comparison. We conduct DSE for both UNPU and LUT TENSOR CORE at the Tensor Core level. Using $W_{INT8}A_{INT2}$ as an example under a Tensor Core configuration of $M \times N \times K = 512$, an ablation study evaluates the impact of each optimization. Table 2 shows that the weight reinterpretation for multi-bit weights and symmetrization enhance compute intensity and power efficiency by 30%. Additional optimizations, including offline weight reinterpretation, negation circuit elimination, DFG transformation, and kernel fusion, enable LUT TENSOR CORE to achieve a 1.44× improvement in these metrics compared to UNPU.

4.5.3 Accelerators for quantized DNN. Previous works, such as Ant [19], FIGNA [25] and Mokey [78], primarily design PEs with MACs for dedicated quantized precision (e.g., int8×int8 or int4×fp16). While efficient for certain data types, these designs lack flexibility in adapting to different precision requirements. They either sacrifice model accuracy when converting to lower precision formats or miss efficiency opportunities when converting to higher precision formats. In contrast, we adopt a LUT-based approach that supports 1-4 bit INT weights and FP/INT 16/8 activations via different LLAMA instructions, covering most low-bit LLM use cases. Table 3 compares LUT TENSOR CORE to other accelerators.

4.6 Software Optimization Analysis

4.6.1 Table precompute fusion analysis. Table 4 demonstrates the impact of incorporating precomputation with the DNN compiler Welder[62], which enhances inference performance by optimizing operator fusion. This evaluation was conducted on a single layer of the OPT-175B, BLOOM-176B, and LLAMA2-70B models in both

Table 2: LUT TENSOR CORE compared with UNPU [38]: $W_{INT2}A_{INT8}$ Tensor Core case.

Configuration	Area (mm ²)	Normalized Compute Intensity	Power (mW)	Normalized Power Efficiency
UNPU (DSE Enabled)	17,271.71	1×	23.39	1×
+ Weight Reinterpretation	13,116.60	1.317×	17.98	1.301×
+ Negation Circuit Elimination	12,780.05	1.351×	17.37	1.347×
+ DFG Trans. + Kernel Fusion				
=LUT TENSOR CORE (Proposed)	11,991.29	1.440×	16.22	1.442×

Table 3: LUT TENSOR CORE compared with accelerators for quantized models.

	UNPU[38]	Ant[19]	Mokey[78]	FIGNA[25]	LUT TENSOR CORE
Act. Format	INT16	flint4	FP16/32, INT4	FP16/32, BF16	FP/INT8, FP/INT16
Wgt. Format	INT1~INT16	flint4	INT3/4	INT4/8	INT1~INT4
Compute Engine	LUT	flint-flint MAC	Multi Counter	Pre-aligned INT MAC	LUT
Process	65nm	28nm	65nm	28nm	28nm
PE Energy Eff.	27TOPs/W @0.9V ($W_{INT1}A_{INT16}$)	N/A	N/A	2.19× FP16-FP16 ($W_{INT4}A_{FP16}$)	63.78TOPs/W @0.9V DC ($W_{INT1}A_{INT8}$)
Compiler Stack	✗	✗	✗	✗	✓
Eval. Models	VGG-16, AlexNet	ResNet, BERT	BERT, Ro/DeBERTa	BERT, BLOOM, OPT	LLAMA, BitNet, BLOOM, OPT

Table 4: Comparison of seperated table precompute and fused table precompute. With operator fusion, the table precompute overhead is negligible.

Model	Config	Welder	Welder +precompute	Welder +Fused precompute
OPT-175B	BS1SEQ2048	32.38 ms	38.77 ms	33.63 ms
OPT-175B	BS1024SEQ1	14.99 ms	17.43 ms	15.50 ms
BLOOM-176B	BS1SEQ4096	107.11 ms	129.85 ms	108.38 ms
BLOOM-176B	BS1024SEQ1	20.99 ms	26.05 ms	21.31 ms
LLAMA2-70B	BS1SEQ4096	34.68 ms	37.60 ms	35.65 ms
LLAMA2-70B	BS1024SEQ1	11.45 ms	15.21 ms	11.75 ms

batch prefilling and decoding configurations. Initially, precomputation on CUDA Cores led to average overhead of 16.47% and 24.41%. However, by treating precomputation as an independent operator within Welder’s search space, overhead is reduced to 2.62% and 2.52%, making it negligible in the overall execution time.

4.6.2 Table quantization analysis. To evaluate the impact of table quantization, we conduct comparative experiments on a LLAMA2-7B model [65] with 2-bit quantized weights. The first data row represents the original $W_{FP16}A_{FP16}$ LLAMA2-7B model, and the second item corresponds to the LLAMA-3B model reported in the BitNet-b1.58 paper [44]. The following 2-bit model is derived from BitDistiller [14], which is an open-source QAT framework to enhance ultra low-bit LLMs. The original configuration comprised INT2 weights and FP16 activations. Building upon the open-source code of BitDistiller, we further implemented INT8 table quantization with LUT-based mpGEMM. The evaluation metrics, aligned with BitDistiller, including perplexity on the WikiText-2 dataset [46], 5-shot accuracy on MMLU [20], and zero-shot accuracy across several tasks [2, 7, 48, 59, 79]. The results of this empirical study are summarized in Table 5. ‘N/A’ in the second data row indicates that the MMLU accuracy is not reported in [44]. Although the 2-bit weight quantization underperforms compared to the original $W_{FP16}A_{FP16}$ LLAMA2-7B model, it still outperforms the $W_{FP16}A_{FP16}$ LLAMA-3B model. Notably, the INT8 table quantization does not compromise

Table 5: Table quantization analysis on LLAMA models.

# Model Config.	WikiText2 PPL ↓	MMLU 5s ↑	Zero-shot Accuracy ↑					
			HS	BQ	OQ	PQ	WGe	Avg.
LLAMA2-7B $W_{FP16}A_{FP16}$ [65]	5.47	45.3	57.1	77.9	31.4	78.0	69.1	62.7
LLAMA-3B $W_{FP16}A_{FP16}$ [44]	10.04	N/A	43.3	61.8	24.6	72.1	58.2	49.7
LLAMA2-7B $W_{INT2}A_{FP16}$ [14]	7.68	30.5	49.2	70.2	25.8	73.8	63.1	56.4
LLAMA2-7B $W_{INT2}A_{LUT_INT8}$ [14]	7.69	30.61	49.2	70.0	26.2	73.7	63.5	56.5

model accuracy, showing a negligible degradation in perplexity and a slight increase in task accuracy, which may be attributed to the regularizing effect of quantization.

5 Discussion and Limitations

Low-Bit Training and Finetuning. Currently, LUT TENSOR CORE is only applicable to inference acceleration for low-bit LLMs. Recent trends show an increasing interest in low-bit training and fine-tuning for LLMs [11, 72]. While LUT TENSOR CORE’s approach for mpGEMM is applicable during the forward pass of low-bit training, the complexity and stability of the training process still demand more high precision computation in the backward pass. This involves tensors and calculations such as gradients and optimizer states, which are not fully compatible with low-bit formats yet. Further, the efficiency of training is impacted by a broad spectrum of factors such as memory efficiency and communication efficiency, beyond GEMM performance. Consequently, optimizing the low-bit training process requires a more comprehensive strategy, possibly entailing new training algorithms that can embrace lower precision and hardware innovations to support the intricate requirements of training workflows. We identify these challenges as potential future directions to extend LUT TENSOR CORE for training.

Long-Context Attention and KV Cache Quantization. Addressing long contexts is an important frontier for LLM capabilities [13, 56]. In long-context scenarios, the attention mechanism often becomes the computational bottleneck. Current research and practice indicate that during the prefilling stage, quantizing attention computation to FP8 does not significantly compromise model

accuracy [60]. However, the effects of ultra-low-bit precision on model accuracy remain largely unexplored. During the decoding phase, several studies have shown that quantizing the KV cache to 4-bit or even 2-bit has a negligible impact on model performance [22, 41]. Given that the Q matrix remains in high precision, the computation aligns with mpGEMM. Exploring LUT TENSOR CORE for long-context scenarios presents a promising direction for future research.

More Data Flexibility and Non-Integer Weights. We believe that the LUT-based method is inherently suited for flexible precision combinations, as it replaces the main dot product operation with table lookups. Currently, LUT TENSOR CORE supports $W_{INT}A_{FP}$ and $W_{INT}A_{INT}$ combinations. To extend this to W_{FP} , our preliminary strategy involves treating the mantissa and sign bit similarly to W_{INT} , using them as table indices. The exponent bits, on the other hand, are treated as inputs to shifters. The LUT approach also accommodates non-integer weight formats. For example, in the case of ternary weights, the LUT approach can pack three ternary weights into 5 bits, whereas ADD-/MAC-based methods require 6 bits to represent the same information.

Emerging Trends in Supporting mpGEMM. Emerging GPUs such as B100 [8] natively support mixed-precision GEMM in Tensor Cores [9, 50]. Blackwell introduces narrow precision formats such as FP4, FP6, FP8, and their variants NVFP4, MXFP4, MXFP6, and MXFP8. It enables a range of mixed precision GEMM, including combinations of $A_{FP4,FP6,FP8} \times W_{FP4,FP6,FP8}$ and $A_{MXF4,MXF6,MXF8} \times W_{MXF4,MXF6,MXF8}$, while providing the same throughput as $W_{FP8}A_{FP8}$ Tensor Cores. LUT TENSOR CORE supports these operations through a bit-serial approach and achieves scalable performance across different formats. With the emergence of native support from major vendors like NVIDIA, mpGEMM is likely to become a critical and widely-adopted computing pattern.

Roofline Analysis of LUT TENSOR CORE. Figure 19 presents a roofline chart for both the conventional $W_{FP16}A_{FP16}$ Tensor Core and the LUT-based $W_{INT1}A_{FP16}$ Tensor Core on an A100 memory system. The x-axis represents operational intensity based on main memory traffic. The area occupied by the $W_{INT1}A_{FP16}$ Tensor Core from LUT TENSOR CORE is only 58.4% of the area of the $W_{FP16}A_{FP16}$ Tensor Core, yet it provides 4 $\times$ the theoretical FLOPs. While the original $W_{FP16}A_{FP16}$ is compute-bound, the naïve LUT-based implementation is memory-bound. Through the software-hardware co-optimization efforts—reinterpreting weights to halve table size and reducing activation memory traffic, employing elongated tiling for better data reuse, and swizzling thread blocks to enhance the L2 hit rate—LUT TENSOR CORE has enhanced operational intensity and pushed the optimized point close to the "ridge point".

6 Related work

Low-Bit DNN Accelerators. As LLMs grow in size, there is an increasing need for low-bit quantization techniques to reduce model size and computational requirements. Hardware accelerators have been developed to efficiently support lower bit-width data types for quantized model inference. NVIDIA's GPU architectures reflect this trend, progressively incorporating lower-precision formats. Starting with the Fermi architecture's support for FP32 and FP64, subsequent architectures have progressively included lower bit-width formats such as FP16 in Pascal, INT4 and INT8 in Turing,

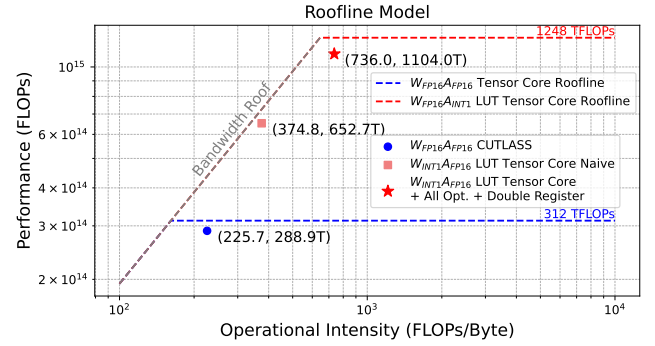


Figure 19: Roofline analysis of conventional $W_{FP16}A_{FP16}$ Tensor Core and $W_{INT1}A_{FP16}$ from LUT TENSOR CORE.

and BF16 in Ampere. In the era of LLMs, Hopper has introduced FP8 [47] and Blackwell has advanced to FP4 [57]. Beyond GPUs, recent studies propose customized accelerators that specifically target low-bit quantized DNNs [19, 35, 43, 58, 77, 78]. Although these advances demonstrate significant progress, they predominantly focus on GEMM operations where both inputs (weights and activations) share the same datatype and bit-width. FIGNA [25] customizes an $W_{INT4}A_{FP16}$ arithmetic unit for enhanced low-bit LLM inference. LUT TENSOR CORE improves the efficiency of mpGEMM with LUT-based computing paradigm, and offers the flexibility to support diverse precision combinations without the need for complex hardware redesigns.

Sparse DNN Accelerators. Alongside low-bit quantization, sparsity is another popular strategy to reduce model size and accelerate DNN inference. Sparsity leverages the inherent zero-valued elements within DNN weight matrices or activations, omitting them from computation and storage to improve efficiency. With the advent of the NVIDIA A100 GPU, Sparse Tensor Cores were introduced, offering native support for sparsity by facilitating 2:4 structured sparsity [6]. Beyond commercial GPUs, there has been a growing interest in customized sparse DNN accelerators. These designs are tailored to exploit sparsity to varying degrees, often employing techniques such as pruning, zero-skipping, and sparse matrix formats to optimize both storage and computation [17, 23, 24, 61, 70, 74, 85]. Sparsity is also prevalent in low-bit LLMs. When combined with quantization, sparsity has the potential to yield even more substantial efficiency gains. However, effectively integrating both quantization and sparsity poses significant challenges in preserving model accuracy and designing efficient microarchitectures. Incorporating sparsity into LUT TENSOR CORE represents a promising research direction, which we leave for future exploration.

7 Conclusion

This paper presents LUT TENSOR CORE, a software-hardware co-design based on a LUT-based computing paradigm to enable efficient mixed-precision GEMM operations for low-bit LLM acceleration. LUT TENSOR CORE enhances performance, provides broad flexibility for various precision combinations, and seamlessly integrates with existing accelerator architectures and software ecosystems.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2019. PIQA: Reasoning about Physical Commonsense in Natural Language. *arXiv:1911.11641 [cs.CL]*
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [4] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. 2024. Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems* 36 (2024).
- [5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: end-to-end optimization stack for deep learning. *arXiv preprint arXiv:1802.04799* 11, 20 (2018).
- [6] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. Nvidia a100 tensor core gpu: Performance and innovation. *IEEE Micro* 41, 2 (2021), 29–35.
- [7] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions. *arXiv:1905.10044 [cs.CL]*
- [8] NVIDIA Corporation. 2025. *NVIDIA Blackwell Architecture Technical Brief*. Technical Report. NVIDIA Corporation. <https://resources.nvidia.com/en-us-blackwell-architecture?ncid=no-ncid>
- [9] NVIDIA Corporation. 2025. Parallel Thread Execution ISA Version 8.8. <https://docs.nvidia.com/cuda/parallel-thread-execution/>. Accessed: 2025-05-02.
- [10] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems* 35 (2022), 30318–30332.
- [11] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems* 36 (2024).
- [12] Tim Dettmers and Luke Zettlemoyer. 2023. The case for 4-bit precision: k-bit inference scaling laws. In *International Conference on Machine Learning*. PMLR, 7750–7774.
- [13] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanxuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. 2024. Longrope: Extending llm context window beyond 2 million tokens. *arXiv preprint arXiv:2402.13753* (2024).
- [14] Dayou Du, Yijia Zhang, Shijie Cao, Jiaqi Guo, Ting Cao, Xiaowen Chu, and Ningyi Xu. 2024. BitDistiller: Unleashing the Potential of Sub-4-Bit LLMs via Self-Distillation. *arXiv preprint arXiv:2402.10631* (2024).
- [15] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [16] ggml.org. 2025. llama.cpp: Port of LLaMA models to C/C++. <https://github.com/ggml-org/llama.cpp>.
- [17] Ashish Gondimalla, Mithuna Thottethodi, and TN Vijaykumar. 2023. Eureka: Efficient Tensor Cores for One-sided Unstructured Sparsity in DNN Inference. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 324–337.
- [18] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2023. Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [19] Cong Guo, Chen Zhang, Jingwen Leng, Zihan Liu, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2022. Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1414–1433.
- [20] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. *arXiv:2009.03300 [cs.CY]*
- [21] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).
- [22] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079* (2024).
- [23] Guyue Huang, Zhengyang Wang, Po-An Tsai, Chen Zhang, Yufei Ding, and Yuan Xie. 2023. RM-STC: Row-Merge Dataflow Inspired GPU Sparse Tensor Core for Energy-Efficient Sparse Acceleration. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 338–352.
- [24] Dongseok Im and Hoi-Jun Yoo. 2024. LUTein: Dense-Sparse Bit-Slice Architecture With Radix-4 LUT-Based Slice-Tensor Processing Units. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 747–759.
- [25] Jaeyong Jang, Yulhwa Kim, Juheun Lee, and Jae-Joon Kim. 2024. FIGNA: Integer Unit-Based Accelerator Design for FP-INT GEMM Preserving Numerical Accuracy. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 760–773.
- [26] Yongkweon Jeon, Baeseong Park, Se Jung Kwon, Byeongwook Kim, Jeongin Yun, and Dongsoo Lee. 2020. Biggemm: matrix multiplication with lookup table for binary-coding-based quantized dnn. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–14.
- [27] Patrick Judd, Jorge Albericio, Tayler Hetherington, Tor M Aamodt, and Andreas Moshovos. 2016. Stripes: Bit-serial deep neural network computing. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
- [28] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [29] Ayush Kaushal, Tejas Vaidhya, Arnab Kumar Mondal, Tejas Pandey, Aaryan Bhagat, and Irina Rish. 2024. Spectra: Surprising effectiveness of pretraining ternary language models at scale. *arXiv preprint arXiv:2407.12327* (2024).
- [30] Mahmoud Khairy, Zhesheng Shen, Tor M Aamodt, and Timothy G Rogers. 2020. Accel-Sim: An extensible simulation framework for validated GPU modeling. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 473–486.
- [31] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. 2023. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629* (2023).
- [32] Tanishq Kumar, Zachary Ankner, Benjamin F Spector, Blake Bordonon, Niklas Muennighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. 2024. Scaling laws for precision. *arXiv preprint arXiv:2411.04330* (2024).
- [33] Andrey Kuzmin, Mart Van Baalen, Yuwei Ren, Markus Nagel, Jorn Peters, and Tijmen Blankevoort. 2022. Fp8 quantization: The power of the exponent. *Advances in Neural Information Processing Systems* 35 (2022), 14651–14662.
- [34] Hyoukjun Kwon, Prasanth Chatarasi, Vivek Sarkar, Tushar Krishna, Michael Pellauer, and Angshuman Parashar. 2020. Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings. *IEEE micro* 40, 3 (2020), 20–29.
- [35] Alberto Delmas Lascorz, Mostafa Mahmoud, Ali Hadi Zadeh, Milos Nikolic, Kareem Ibrahim, Christina Giannoula, Ameer Abdelhadi, and Andreas Moshovos. 2024. Atalanta: A Bit is Worth a “Thousand” Tensor Values. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 85–102.
- [36] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2023. Bloom: A 176b-parameter open-access multilingual language model. (2023).
- [37] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. 2023. Owq: Lessons learned from activation outliers for weight quantization in large language models. *arXiv preprint arXiv:2306.02272* (2023).
- [38] Jinmook Lee, Changhyeon Kim, Sanghoon Kang, Dongjo Shin, Sangyeob Kim, and Hoi-Jun Yoo. 2019. UNPU: An Energy-Efficient Deep Neural Network Accelerator With Fully Variable Weight Bit Precision. *IEEE Journal of Solid-State Circuits* 54, 1 (2019), 173–185. <https://doi.org/10.1109/JSSC.2018.2865489>
- [39] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978* (2023).
- [40] Jing Liu, Ruihao Gong, Xiuying Wei, Zhiwei Dong, Jianfei Cai, and Bohan Zhuang. 2024. QLLM: Accurate and Efficient Low-Bitwidth Quantization for Large Language Models.
- [41] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750* (2024).
- [42] Zechun Liu, Changsheng Zhao, Hanxian Huang, Sijia Chen, Jing Zhang, Jiawei Zhao, Scott Roy, Lisa Jin, Yunyang Xiong, Yangyang Shi, et al. 2025. ParetoQ: Scaling Laws in Extremely Low-bit LLM Quantization. *arXiv preprint arXiv:2502.02631* (2025).
- [43] Yun-Chen Lo and Ren-Shuo Liu. 2023. Bucket Getter: A Bucket-based Processing Engine for Low-bit Block Floating Point (BFP) DNNs. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (<conf-loc>, <city>-Toronto/<city>, <state>-ON/<state>, <country>-Canada/<country>, </conf-loc>)* (MICRO '23). Association for Computing Machinery, New York, NY, USA, 1002–1015. <https://doi.org/10.1145/3613424.3614249>
- [44] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. 2024. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. *arXiv preprint arXiv:2402.17764* (2024).

- [45] Saeed Maleki. 2023. Look-Up mAI GeMM: Increasing AI GeMMs Performance by Nearly 2.5 x via msGeMM. *arXiv preprint arXiv:2310.06178* (2023).
- [46] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. *arXiv:1609.07843 [cs.CL]*
- [47] Paulius Mikićevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. 2022. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433* (2022).
- [48] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. *arXiv:1809.02789 [cs.CL]*
- [49] Pranav Nair, Puranjay Datta, Jeff Dean, Prateek Jain, and Aditya Kusupati. 2025. Matryoshka Quantization. *arXiv preprint arXiv:2502.06786* (2025).
- [50] NVIDIA. 2025. CUTLASS: CUDA Templates for Linear Algebra Subroutines. <https://github.com/NVIDIA/cutlass>.
- [51] NVIDIA. 2025. TensorRT-LLM: High-Performance Inference for Large Language Models. <https://github.com/NVIDIA/TensorRT-LLM>. Accessed: 2025-05-02.
- [52] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A Ying, Anurag Mukkara, Rangharajan Venkatesan, Bruce Khailany, Stephen W Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 304–315.
- [53] Gunho Park, Baeseong Park, Minsub Kim, Sungjae Lee, Jeonhoo Kim, Beom-seok Kwon, Se Jung Kwon, Byeongwook Kim, Youngjoo Lee, and Dongsoo Lee. 2023. LUT-GEMM: Quantized Matrix Multiplication based on LUTs for Efficient Inference in Large-Scale Generative Language Models. *arXiv preprint arXiv:2206.09557* (2023).
- [54] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2023. Splitwise: Efficient generative llm inference using phase splitting. *Power* 400, 700W (2023), 1–75.
- [55] David Patterson, Joseph Gonzalez, Urs Holzle, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David R So, Maud Texier, and Jeff Dean. 2022. The carbon footprint of machine learning training will plateau, then shrink. *Computer* 55, 7 (2022), 18–28.
- [56] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071* (2023).
- [57] Bitá Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. 2023. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537* (2023).
- [58] Sungju Ryu, Hyungjun Kim, Wooseok Yi, Eunhwan Kim, Yulhwa Kim, Taesu Kim, and Jae-Joon Kim. 2022. BitBlade: Energy-efficient variable bit-precision hardware accelerator for quantized neural networks. *IEEE Journal of Solid-State Circuits* 57, 6 (2022), 1924–1935.
- [59] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2019. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. *arXiv:1907.10641 [cs.CL]*
- [60] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. *arXiv preprint arXiv:2407.08608* (2024).
- [61] Man Shi, Vikram Jain, Antony Joseph, Maurice Meijer, and Marian Verhelst. 2024. BitWave: Exploiting Column-Based Bit-Level Sparsity for Deep Learning Acceleration. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 732–746.
- [62] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. 2023. Welder: Scheduling Deep Learning Memory Access via Tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 701–718.
- [63] Synopsys Inc. 2018. *Design Compiler User Guide*.
- [64] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [65] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shriti Bhoale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [66] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [67] Oreste Villa, Daniel Lustig, Zi Yan, Evgeny Bolotin, Yaosheng Fu, Niladri Chatterjee, Nan Jiang, and David Nellans. 2021. Need for speed: Experiences building a trustworthy system-level gpu simulator. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 868–880.
- [68] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453* (2023).
- [69] Lei Wang, Lingxiao Ma, Shijie Cao, Quanlu Zhang, Jilong Xue, Yining Shi, Ningxin Zheng, Ziming Miao, Fan Yang, Ting Cao, et al. 2024. Ladder: Enabling Efficient {Low-Precision} Deep Learning Computing through Hardware-aware Tensor Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 307–323.
- [70] Yang Wang, Chen Zhang, Zhiqiang Xie, Cong Guo, Yunxin Liu, and Jingwen Leng. 2021. Dual-side sparse tensor core. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1083–1095.
- [71] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2024. T-mac: Cpu renaissance via table lookup for low-bit llm deployment on edge. *arXiv preprint arXiv:2407.00088* (2024).
- [72] Haocheng Xi, Changhao Li, Jianfei Chen, and Jun Zhu. 2023. Training transformers with 4-bit integers. *Advances in Neural Information Processing Systems* 36 (2023), 49146–49168.
- [73] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
- [74] Jianxun Yang, Zhao Zhang, Zhuangzhi Liu, Jing Zhou, Leibo Liu, Shaojun Wei, and Shouyi Yin. 2021. Fusekna: Fused kernel convolution based accelerator for deep neural networks. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 894–907.
- [75] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems* 35 (2022), 27168–27183.
- [76] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. 2024. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652* (2024).
- [77] Ali Hadi Zadeh, Isak Edo, Omar Mohamed Awad, and Andreas Moshovos. 2020. GOGO: Quantizing Attention-Based NLP Models for Low Latency and Energy Efficient Inference. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE. <https://doi.org/10.1109/micro50266.2020.00071>
- [78] Ali Hadi Zadeh, Mostafa Mahmoud, Ameer Abdelhadi, and Andreas Moshovos. 2022. Mokey: enabling narrow fixed-point inference for out-of-the-box floating-point transformer models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA '22)*. ACM. <https://doi.org/10.1145/3470496.3527438>
- [79] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence? *arXiv:1905.07830 [cs.CL]*
- [80] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
- [81] Yijia Zhang, Lingran Zhao, Shijie Cao, Wenqiang Wang, Ting Cao, Fan Yang, Mao Yang, Shanghang Zhang, and Ningyi Xu. 2023. Integer or floating point? new outlooks for low-bit quantization on large language models. *arXiv preprint arXiv:2305.12356* (2023).
- [82] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 863–879. <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [83] Size Zheng, Siyuan Chen, Siyuan Gao, Liancheng Jia, Guangyu Sun, Runsheng Wang, and Yun Liang. 2023. TileFlow: A Framework for Modeling Fusion Dataflow via Tree-based Analysis. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1271–1288.
- [84] Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, et al. 2022. {ROLLER}: Fast and efficient tensor compilation for deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 233–248.
- [85] Maohua Zhu, Tao Zhang, Zhenyu Gu, and Yuan Xie. 2019. Sparse tensor core: Algorithm and hardware co-design for vector-wise sparse neural networks on modern gpus. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 359–371.